CI/CD Pipeline Troubleshoot AWS Case Study

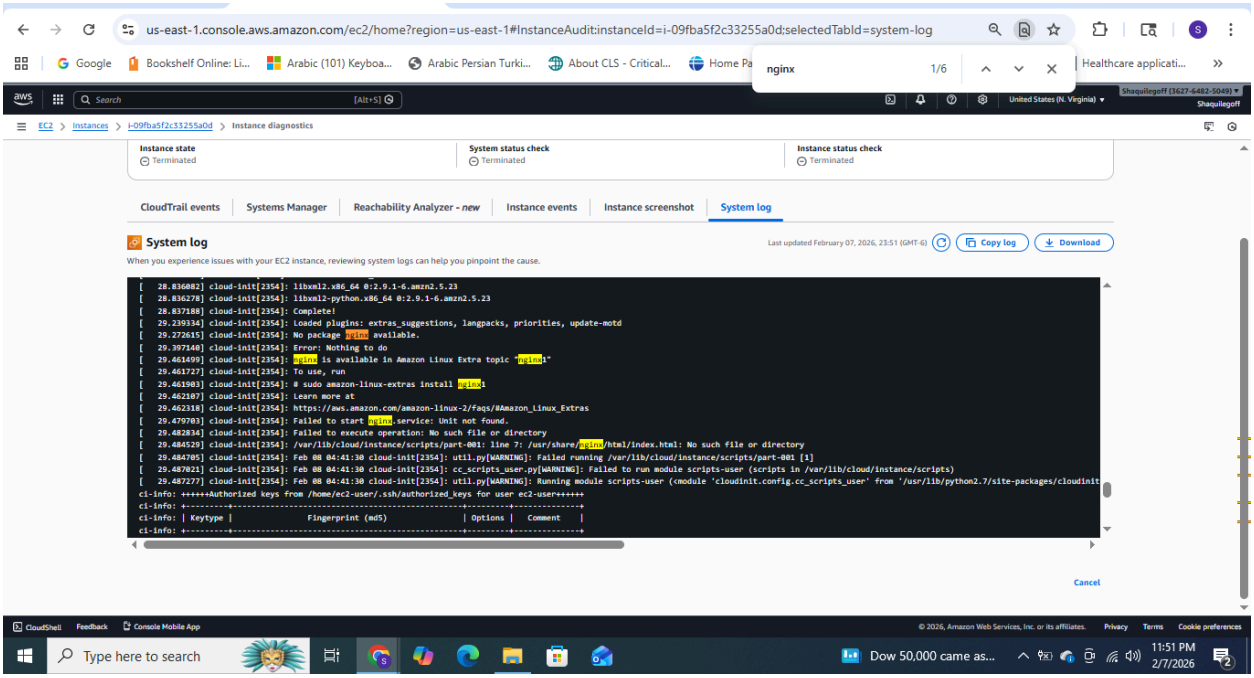
Created by Shaquile Goff

Objective: To demonstrate knowledge of troubleshooting common CI/CD pipeline issues in a work environment. I will present problems within various stages of the pipeline creation process and focus on key components such as IAM roles, configuration errors in YAML, and analyze build logs to correct the problems and ensure successful pipeline execution.

In this project I will highlight the relevant skills used, a step by step guide to how I completed it, challenges over came, and a conclusion of things I learned.

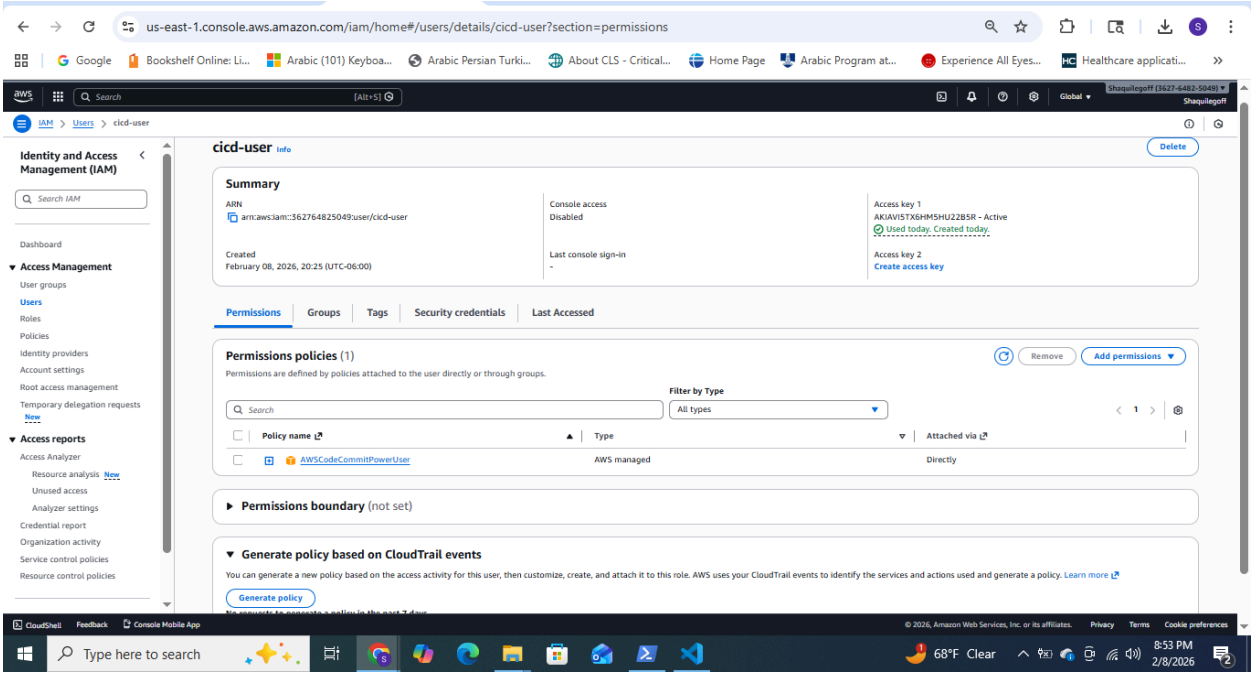
Skills utilized:
CodeCommit
CodeBuild
CodePipeline
IAM
Git
YAML
Powershell
IaaS
PaaS
Log analysis

Steps:
First, I needed to deploy EC2 test servers with desired configuration. I decided to do automate this process through Terraform.
Initial EC2 test deployment via Terraform failed. Troubleshooting via cloud-init logs revealed nginx was misconfigured, causing a server launch failure. I then made adjustments in terraform and the deployment succeeded.



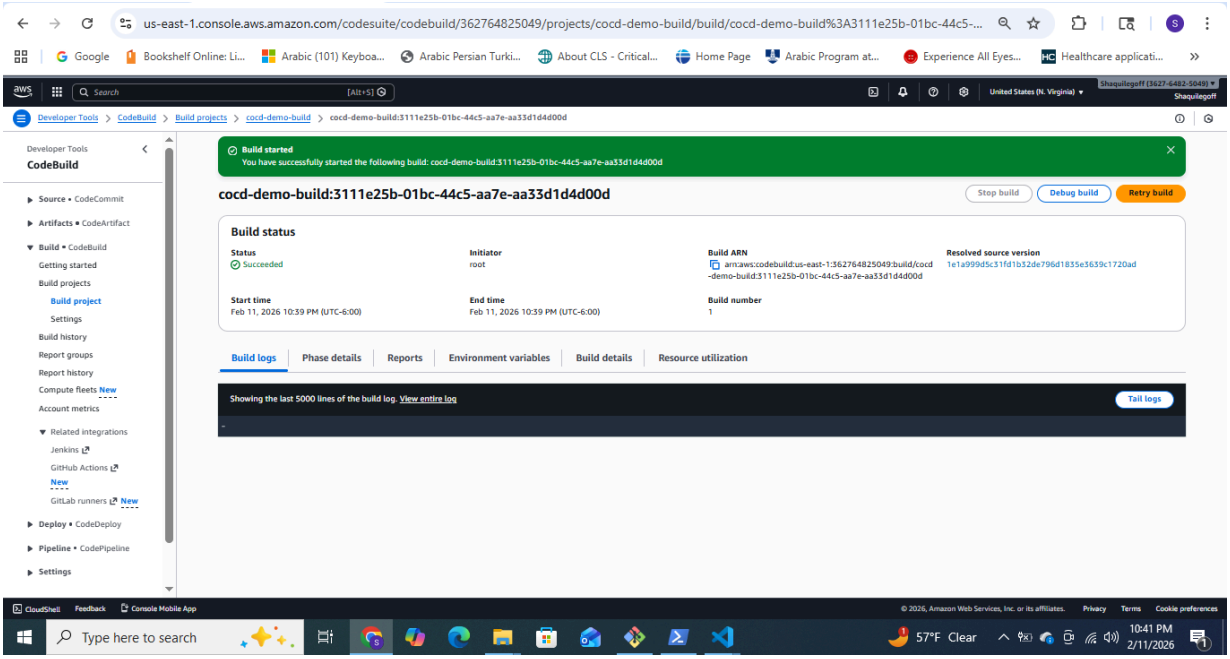
Next, I needed to build the CI/CD pipeline. For this, I utilized AWS CodeBuild, CodeCommit, and CodePipeline. I created code on a local repo and pushed the code to CodeBuild via Git in order to automate the building and provisioning of the necessary cloud infrastructure. I also needed to configure an IAM user to work with CodeCommit.

IAM user created to manage/deploy CodeCommit and follow least privilege best practice. AWS CodeCommit service role created.



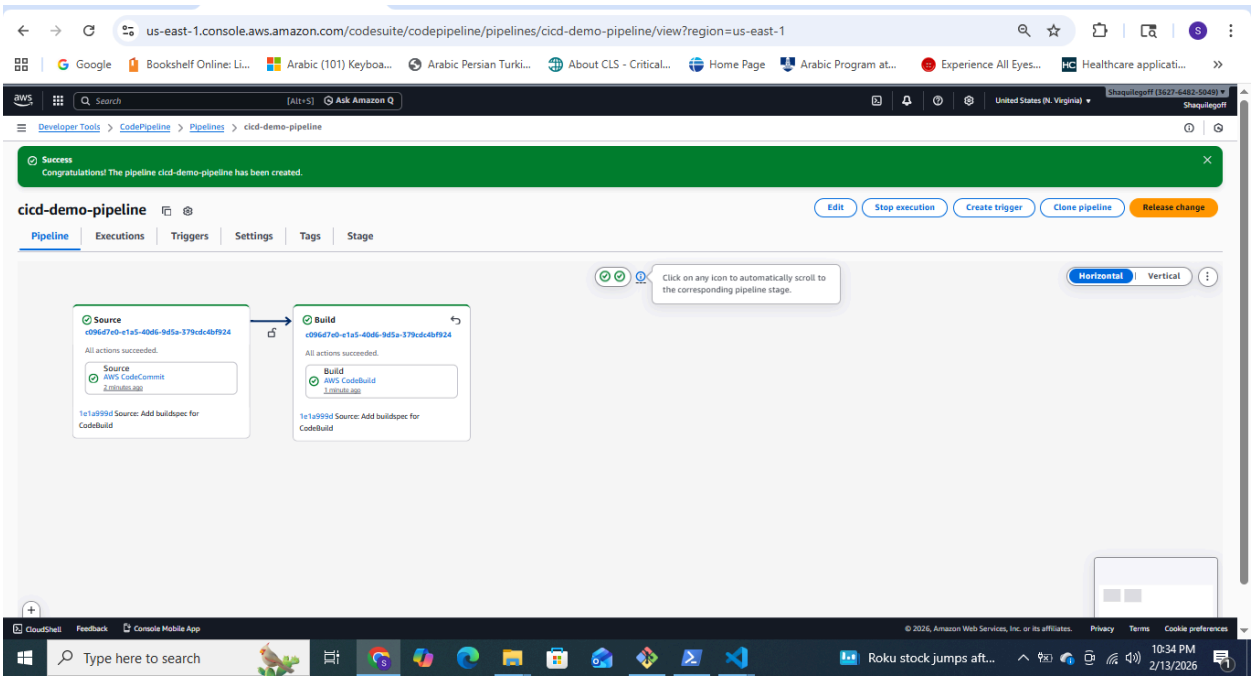
Next, I utilized AI coding assistants to create boilerplate infrastructure code and manually verified to improve deployment efficiency. I then pushed the code and buildspec file to CodeCommit from the local repo.

Successful CodeBuild project created after pushing code from CodeCommit repo.

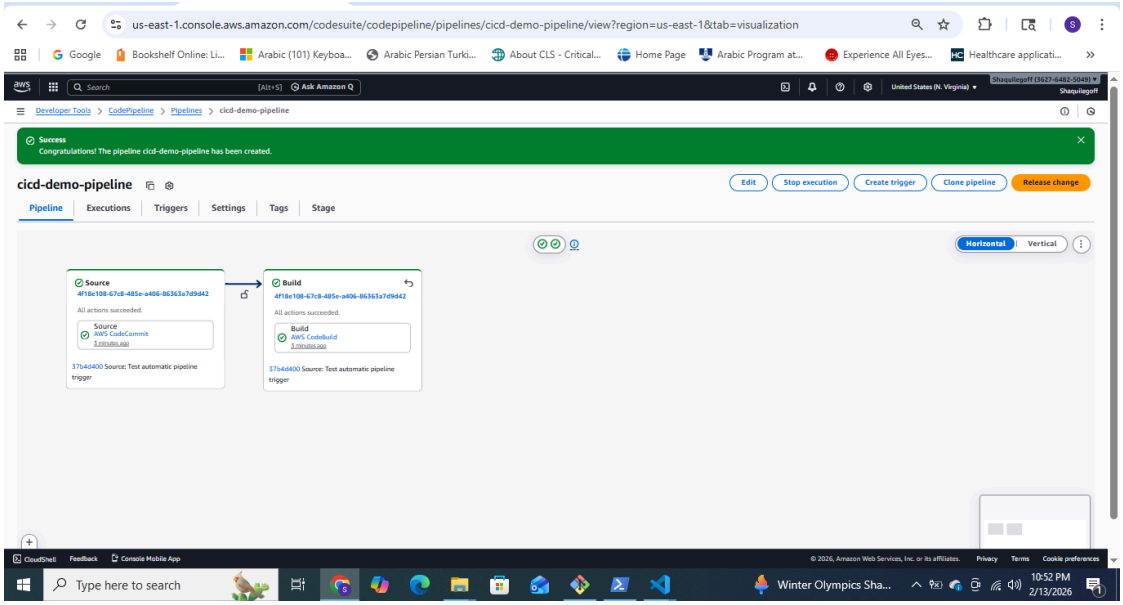


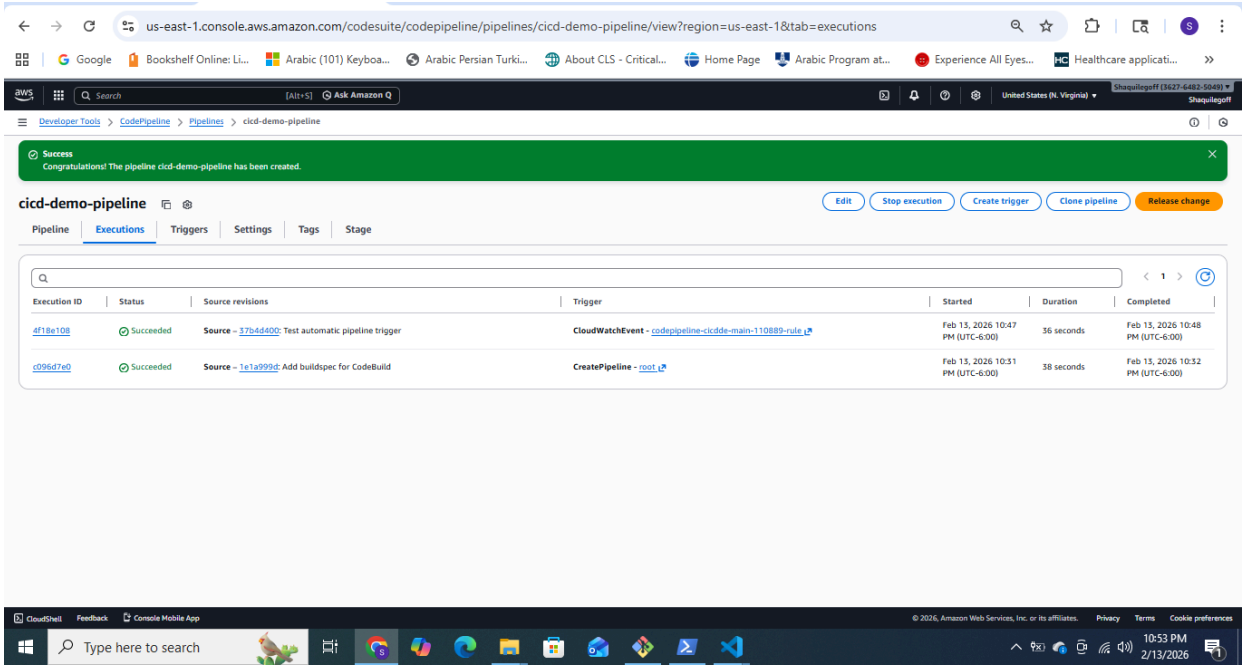
Next, I created a pipeline in CodePipeline to automate the building, testing, and deployment of my new infrastructure.

CodePipeline created successfully

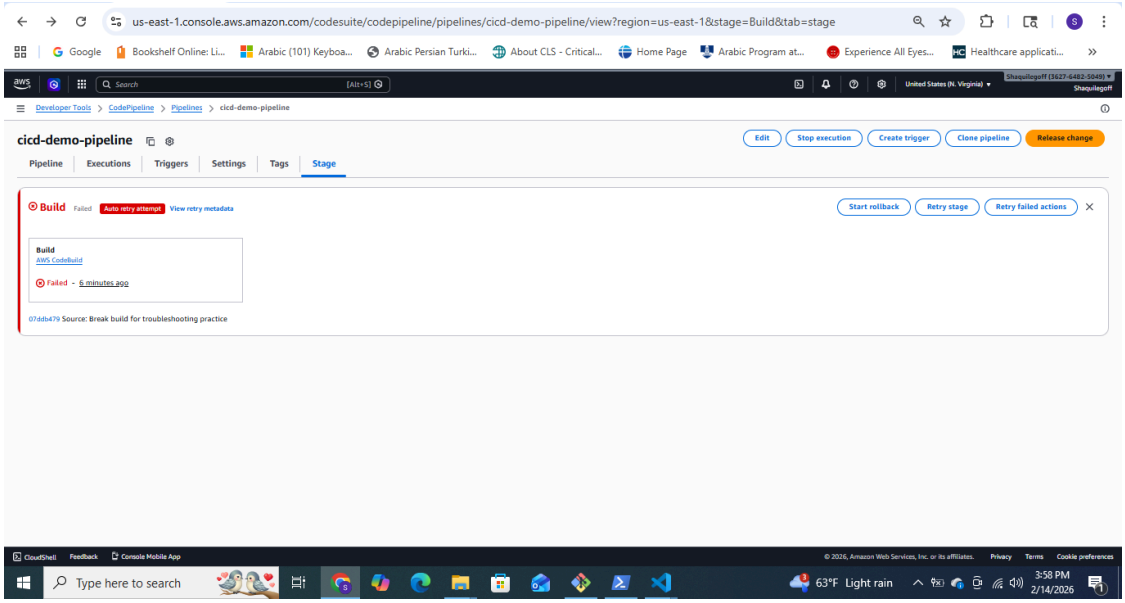


Successful test of CI/CD pipeline by making superficial changes using Git push, verified through CodePipeline

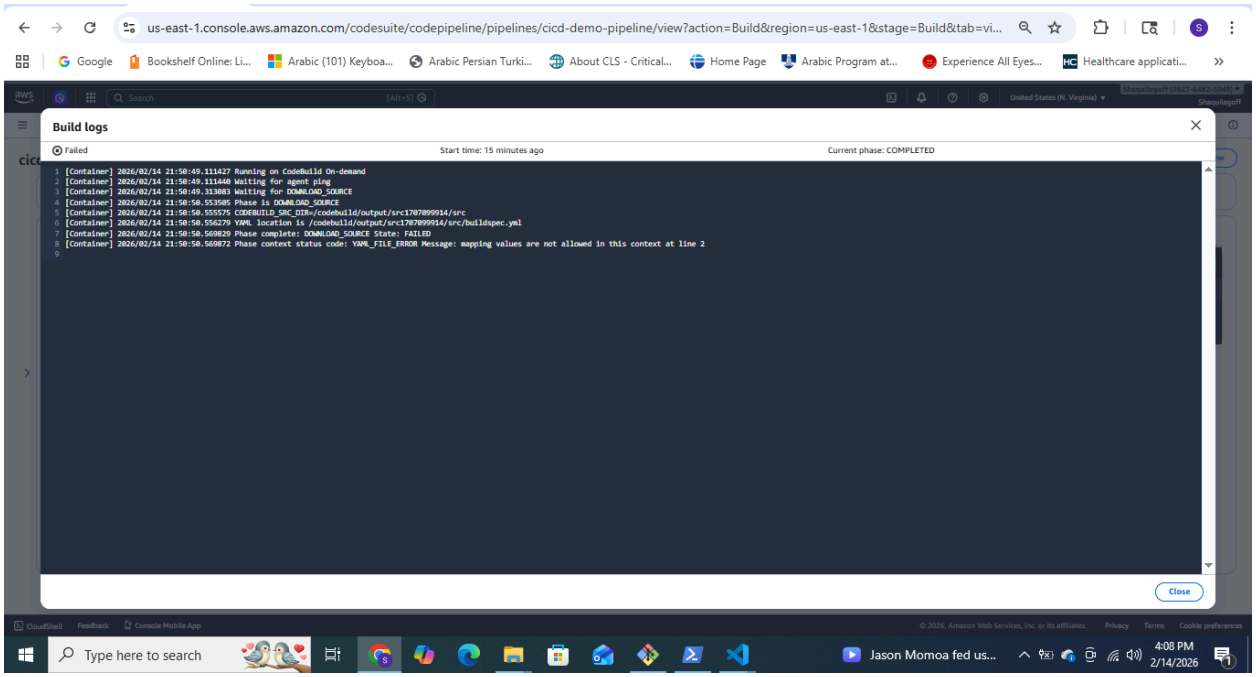




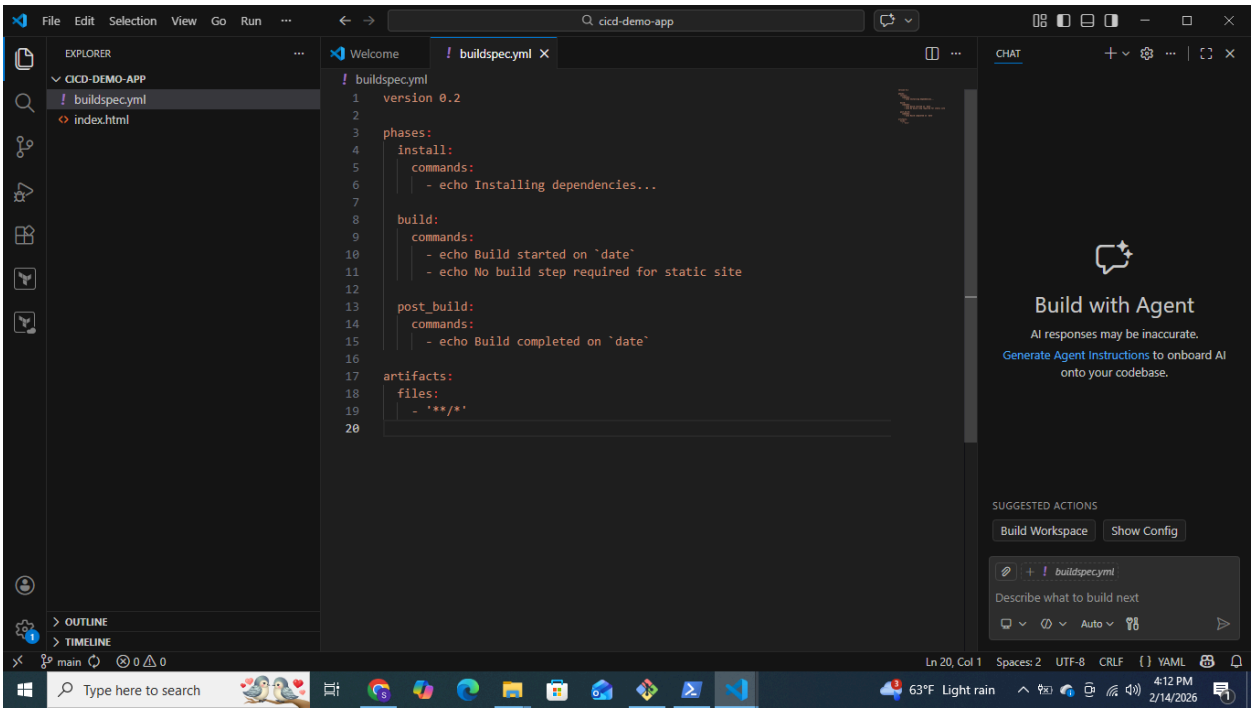
Once the CodePipeline was successfully running, I intentionally introduced an error to disrupt the CI/CD workflow.
Intentionally breaking CI/CD pipeline:

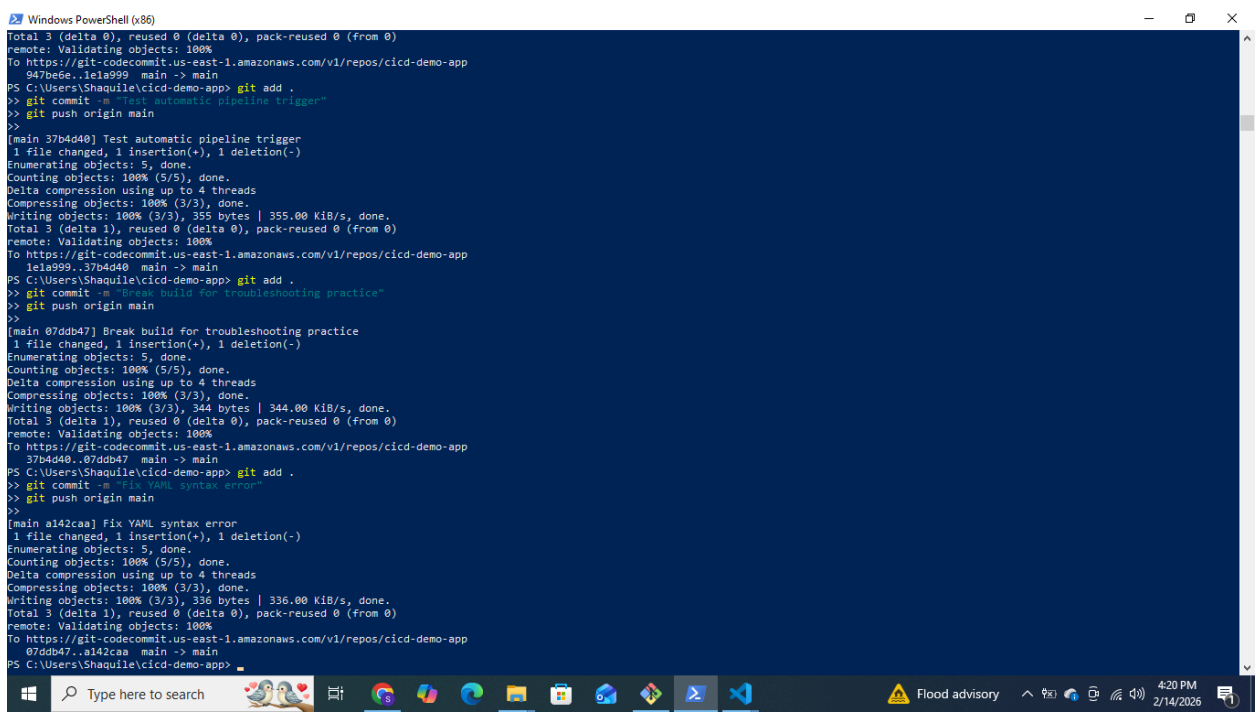


After the Pipeline indicated an error, I went into the cloud build log to see if I can determine where the problem was located. I inspected the build log and noticed “[Container] 2026/02/14 21:50:50.569872 Phase context status code: YAML_FILE_ERROR Message: mapping values are not allowed in this context at line 2” This indicated the error involved the buidspec file, and involved an error with syntax. In a real-world situation, I would look at issues with spacing/indentation or improper key-value formatting.

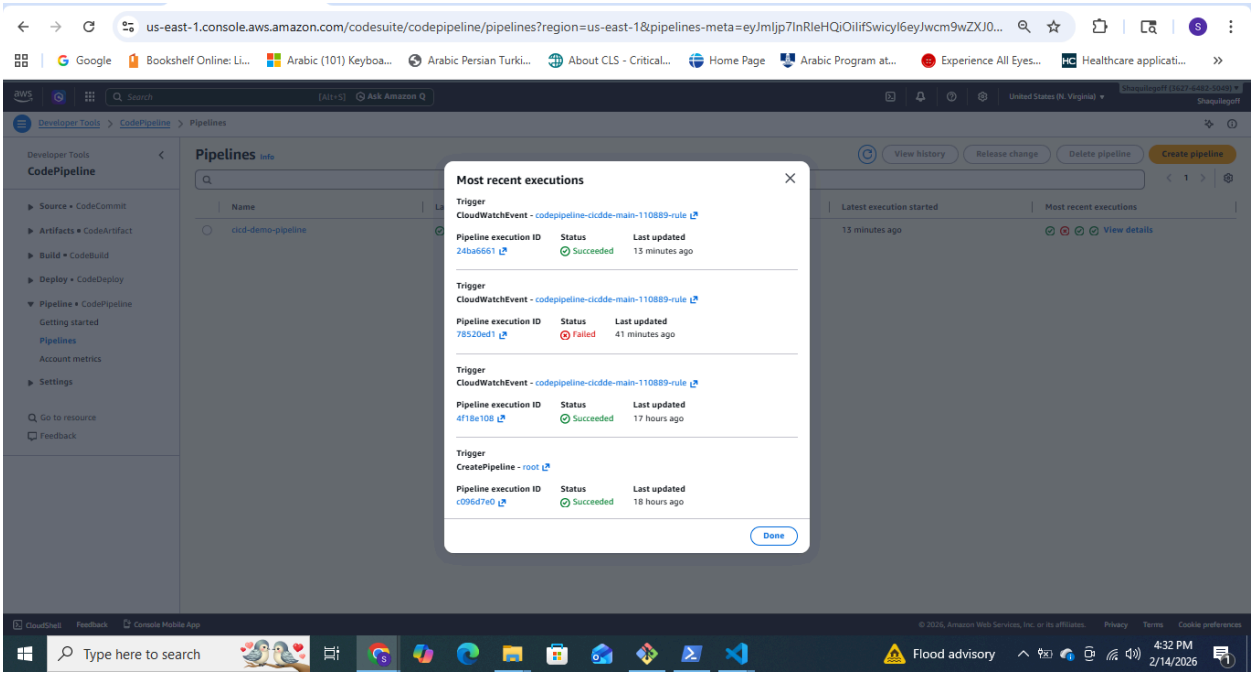


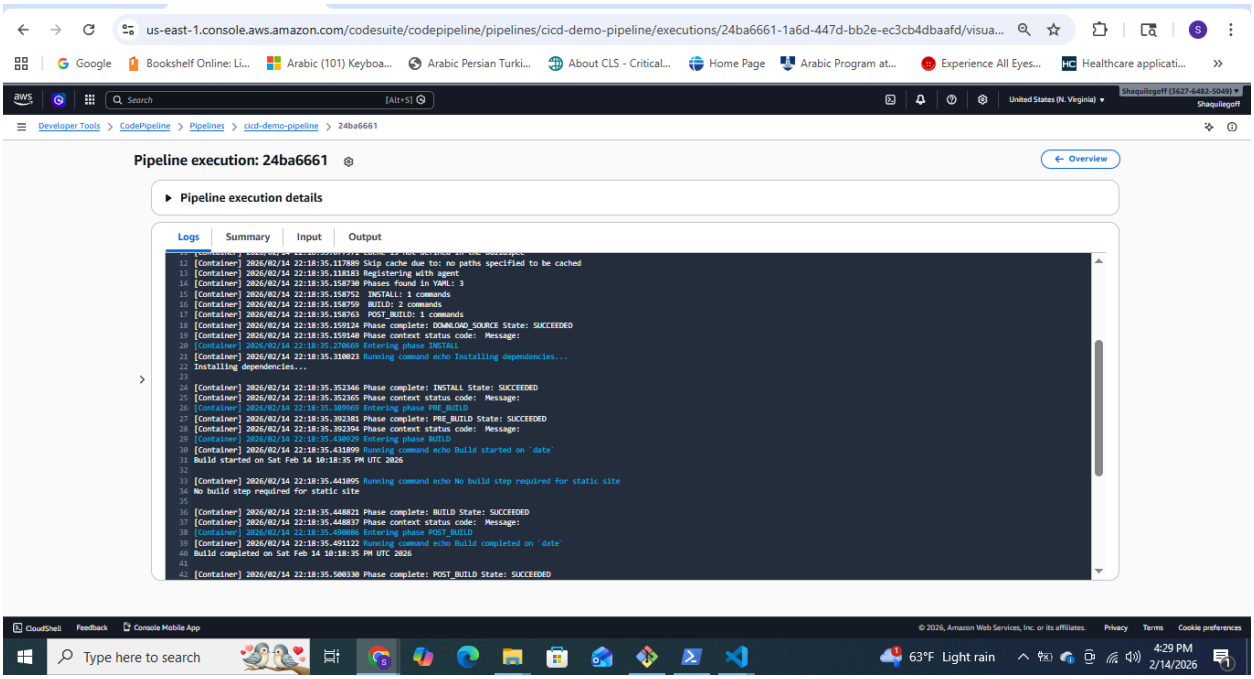
I inspected the YAML file and noticed a syntax error in line 1. The missing colon was causing YAML to not register the key-value.



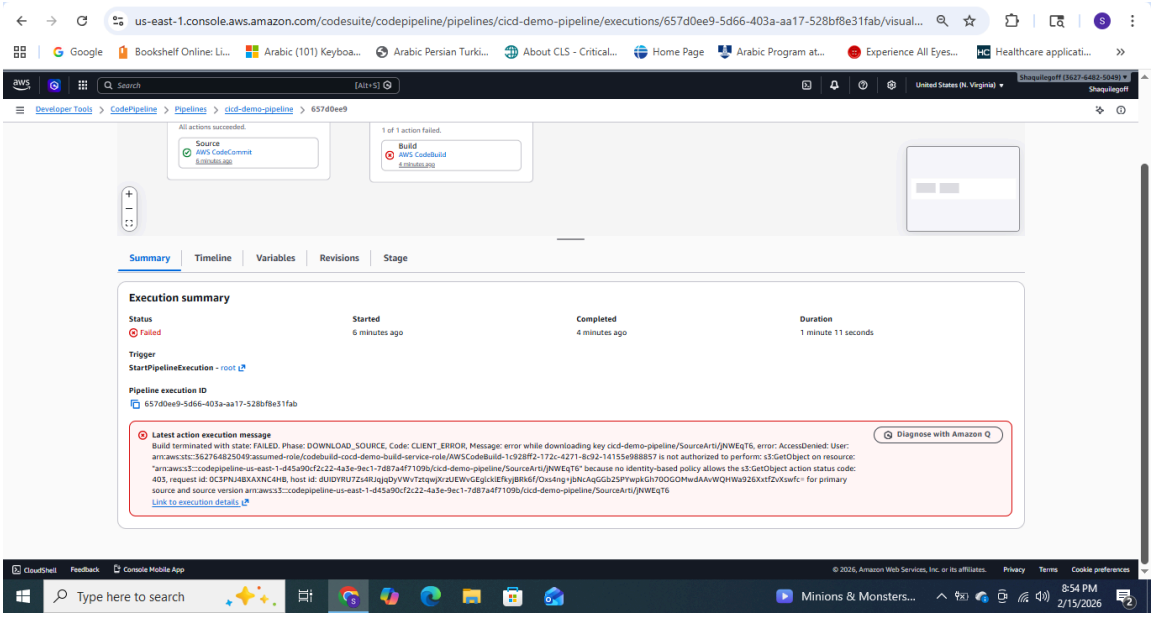


After fixing the YAML error, I pushed the revisions through Git and updated the pipeline successfully. I verified the successful status by examining the revised build logs.





Next, I wanted to simulate another common problem in a work environment: IAM permission configuration issues. After editing the base policy JSON file and attempting to relaunch the pipeline, I encountered a failed pipeline execution.

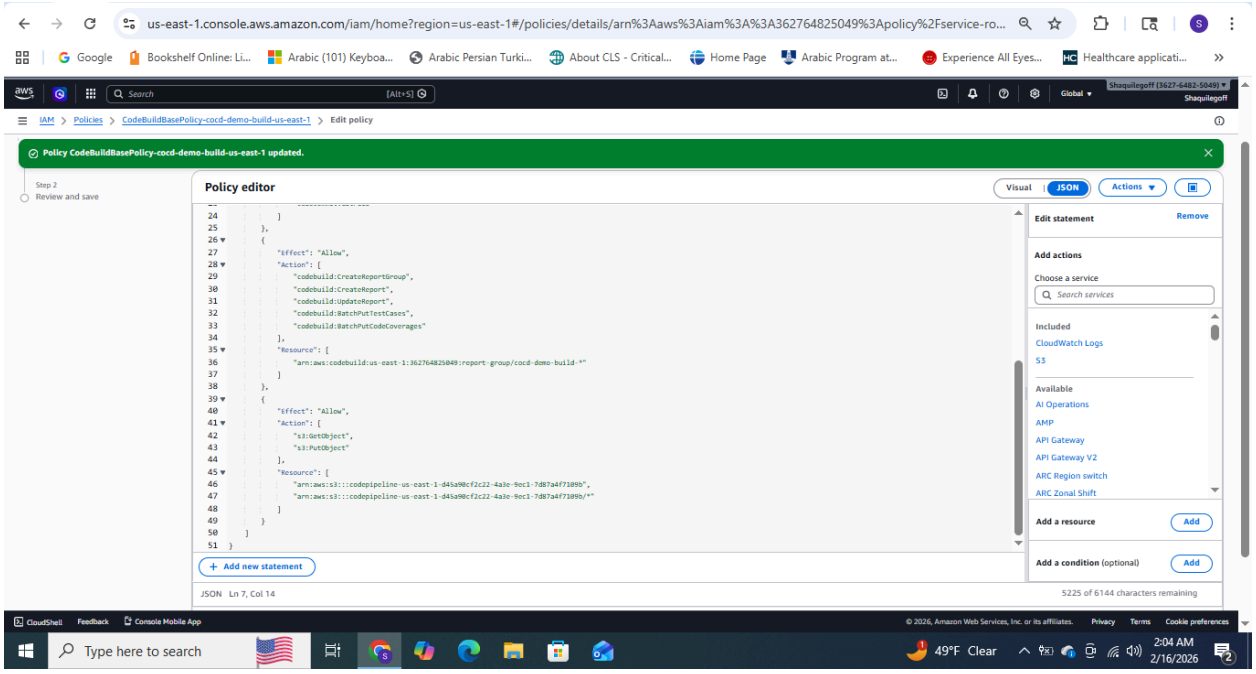


Analyzing the error message allows me to identify the issue with the pipeline launch:

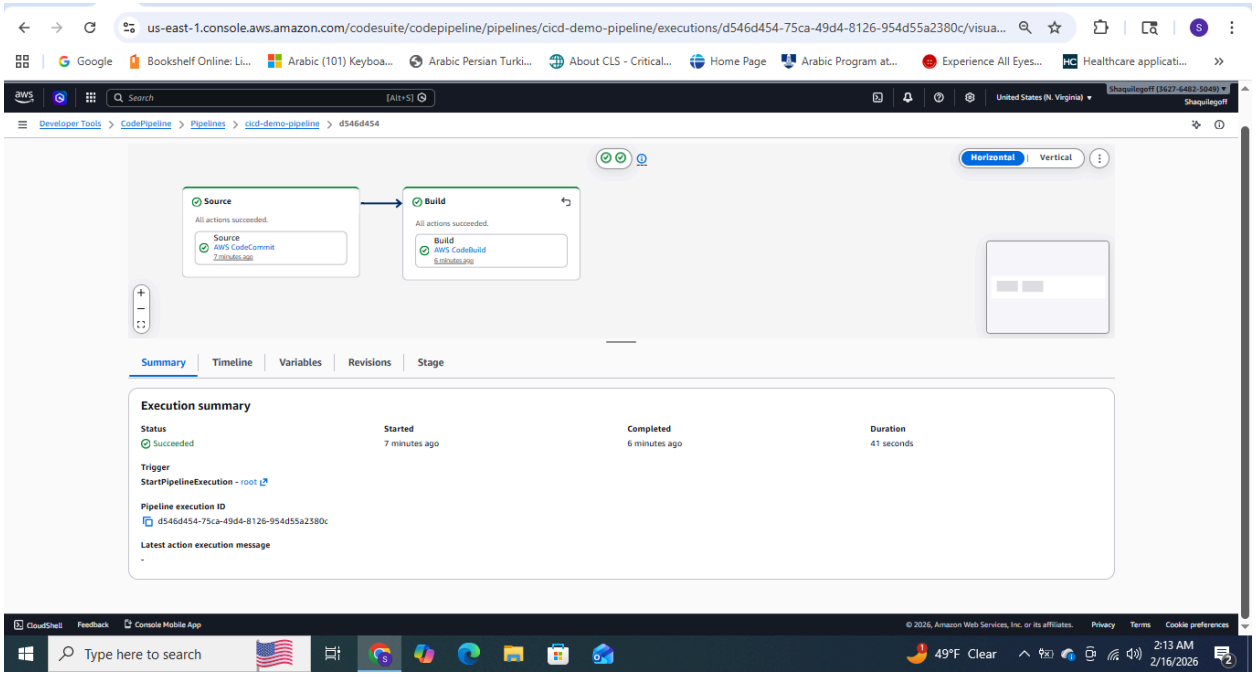
Build terminated with state: FAILED. Phase: DOWNLOAD_SOURCE, Code: CLIENT_ERROR, Message: error while downloading key cicd-demo-pipeline/SourceArti/jNWEqT6, error: AccessDenied: User: arn:aws:sts::362764825049:assumed-role/codebuild-cocd-demo-build-service-role/AWSCodeBuild-1c928ff2-172c-4271-8c92-14155e988857 is not authorized to perform: s3:GetObject on resource: "arn:aws:s3:::codepipeline-us-east-1-d45a90cf2c22-4a3e-9ec1-7d87a4f7109b/cicd-demo-pipeline/SourceArti/jNWEqT6" because no identity-based policy allows the s3:GetObject action status code: 403, request id: 0C3PNJ4BXAXNC4HB, host id: dUIDYRU7Zs4RJqjqDyVWvTztqwjXrzUEWvGEglcklEfkyjBRk6f/Oxs4ng+jbNcAqGGb2SPYwpkGh70OGOMwdAAvWQHWa926XxtfZvXswfc= for primary source and source version arn:aws:s3:::codepipeline-us-east-1-d45a90cf2c22-4a3e-9ec1-7d87a4f7109b/cicd-demo-pipeline/SourceArti/jNWEqT6

The highlighted portion shows the issue lies in missing S3 permissions, preventing the codebuild-cocd-demo-build-service-role from having access to the S3 bucket and its contents. Now that I've identified the problem, I can work on resolving it.

Adding S3 permissions to the base policy enables CodePipeline to acess the S3 bucket and retrieve the necessary files for cloud deployment.



The pipeline was succesfully executed!



Challenges overcome: During the course of this project, I encountered and overcame common cloud troubleshooting issues. Learning to refine YAML syntax and ensuring proper Terraform configuration was crucial to building and automating the infrastructure for this project. In addition, I needed to know how to configure IAM roles, users, and permissions to ensure the different elements of the CI/CD pipeline were able to access appropriate resources. Finally, the ability to read and analyze logs enables efficient and accurate debugging, allowing the CI/CD pipeline to execute properly.

Conclusion: I intended to use this project to showcase my knowledge of common cloud support troubleshooting scenarios in an AWS environment. Using various services in AWS alongside Terraform I built a simple CI/CD pipeline. Afterwards, Introduced problems I believe would be encountered in real world working environments. A knowledge of these problems and their solutions allows me to contribute to keeping cloud infrastructure functioning at the enterprise level, ensuring continued company operation.